Orbit Wars — v7: BC warm-start + guardrailed PPO, a scaling-validated training recipe (*diff* against `rl_math_v6.tex`)

2026-06-09

How to read this. This document is a *delta* on top of `rl_math_v6.tex` (which is itself a delta on `rl_math.tex`). The simulator, the gated-Dirichlet allocation action space, the potential-based reward, PopArt, and the Elo/PFSP league are all unchanged from v6 and are not restated. v7 answers one question: *what training method + architecture is demonstrably stable and improvable at small scale on an RTX 3070 Ti, such that the same recipe can be scaled on an A100 with high confidence?* The answer has three parts: (i) a **behavior-cloning (BC) warm-start** that expresses the strongest scripted bot directly in the gated-allocation action space and eliminates the from-scratch bootstrap failure entirely; (ii) three **PPO stability guardrails** (per-minibatch KL hard stop, linear LR warmup, critic-only warmup) that remove the catastrophic first-update collapse; (iii) a **measured compute model** (steps/s, $/iter, ms/move at deploy) that sizes the A100 run and proves the deployment budget is nowhere near binding.

One-line map.

component	status in v7	where
simulator / parity / comets	unchanged (**see `rl_math_v6.tex` § sys**)	—
gate (Bernoulli fire) + decode	unchanged (**see `rl_math_v6.tex` § dist**); grad-vanish fix **verified**	§2
WHERE distribution	**replaced: categorical** (Dirichlet kept for A/B)	§4
entropy bonus	**replaced: gate-only**	§5
potential shaping + PopArt	unchanged (**see `rl_math_v6.tex` § reward**)	—
Elo/PFSP league	unchanged (**see `rl_math_v6.tex` § league**)	—
bootstrap	**new: BC warm-start**	§3
PPO update	**new: 3 guardrails**	§6
evidence / scaling ladder	**new: measured**	§7
deploy + A100 compute model	**new: measured**	§8

Method, in one paragraph. Train a per-planet-token transformer with two factored heads — a per-source *categorical* over destinations (WHERE) and a per-source Bernoulli launch gate (FIRE) — by (1) behavior-cloning the strongest scripted bot directly into those heads (2–3 minutes of GPU time), then (2) guardrailed PPO against the Elo/PFSP league. Every piece of this was forced by a measured failure of the alternative, documented below.

1 Why the bootstrap, not the algorithm, was the wall

Every from-scratch run in this project's history — GRPO (**see `rl_math.tex` § "GRPO"** (`sec:grpo`)), PPO+GAE, the v5 target actor, the v6 gated allocation — hit the same two walls, reproduced one final time as a controlled 30-iteration smoke (256-dim/2-layer transformer, B=64, T=300, 3070 Ti):

1. **Catastrophic first update.** At iteration 1 the measured update KL was 2.06 with clip-fraction 0.98: *every* sample was clipped, i.e. the update was pure surrogate-boundary drift. The launch rate fell from 13.5 to 1.0 launches/step in that single iteration — the fresh policy's exploratory aggression was annihilated before the critic could attribute credit. The existing end-of-epoch KL check (**see `rl_math_v6.tex` § ppo**) fires only *after* a full epoch (64 minibatch optimizer steps) has already run away.

2. **Passivity plateau.** After the slam, the policy stabilizes at ∼1 launch/step, beats the random anchor ($\approx$ 0.8 win rate) and stalls: policy loss $\approx$ 0, KL $\approx 10^{-3}$, and the fixed-opponent gauntlet (starter/intermediate/greedy) reads 0.000 for all 30 iterations. This is the same basin documented for v4/v5; reward re-shaping moved which basin we landed in but never removed it. The best from-scratch result ever obtained here was a 0.47 win rate vs. starter after 1400 iterations ($\approx$ 20 h on the 3070 Ti).

The conclusion that drives v7: *the optimization machinery is fine; the starting point is fatal.* The fix is to start PPO from a policy that already commits, aims, and wins against the scripted ladder — which the gated-allocation action space makes trivially expressible (§3) — and to make the first PPO updates structurally incapable of destroying it (§6).

2 Gradient-vanish fix: verified

v6's gradient collapse was caused by applying the deploy-time gate deadzone $p = \mathrm{clip}\big((\sigma(z) - 0.2)/0.6,\ 0,\ 1\big)$ during training: clip has zero gradient outside the band, and as gate logits sharpen, both the gate head and (through the fire $\cdot \log \pi_{\text{where}}$ coupling) the WHERE head starve. The fix (already in the notebook, `GatedAllocDist(deploy=False)`: training uses smooth $p = \sigma(z)$ clamped to $[\varepsilon, 1-\varepsilon]$; deployment keeps the hard trim; the greedy fire boundary $p \geq 0.5 \Leftrightarrow z \geq 0$ is identical in both) is now **empirically verified**: over the 30-iteration smoke the gradient norm stayed in [0.7, 5.4] with no downward trend, where the pre-fix run decayed monotonically toward zero.

3 BC warm-start in the gated-allocation space

3.1 The expert, expressed natively in the action space

The strongest scripted anchor is `medium` (nearest non-owned planet, lead-aimed, fire when $\lfloor s/2 \rfloor \geq 20$). Its decision structure maps *exactly* onto the two gated-alloc heads — this is the payoff of the v6 action-space design:

$$\text{fire}^*_s = \mathbb{1}\big[\text{owned}_s \wedge \text{alive}_s \wedge \lfloor S_s/2 \rfloor \geq 20 \wedge \exists\, \text{target}\big], \qquad \text{where}^*_s = \text{onehot}\Big(\arg \min_{d\,:\,\text{non-owned}} \|x_d - x_s\|\Big).$$

Two deliberate deviations from the literal bot: (i) the gated decode dispatches the *full* garrison on fire (the action space cannot express "send half"), so the clone is **"full-send medium"** — measured *stronger* than its teacher (Table 1), consistent with the project-wide lesson that expansion-race

aggression dominates; (ii) expert targets occluded by the sun are dropped from the WHERE loss (the clone learns "medium minus sun-suicide").

3.2 Loss: native to the distribution, not a surrogate

With p_s the smooth gate fire-probability and ℓ_{sd} the masked WHERE logits (alive + reachability + no-self, the same `glogits` the sharpened-softmax greedy uses):

$$\mathcal{L}_{\mathrm{BC}} = \underbrace{-\tfrac{1}{|O|}\sum_{s\in O}\Big[\mathrm{fire}^*_s \log p_s + (1-\mathrm{fire}^*_s)\log(1-p_s)\Big]}_{\text{gate BCE, all owned sources}} \underbrace{-\tfrac{1}{|F|}\sum_{s\in F}\log \mathrm{softmax}(\ell_{s\cdot})_{d^*_s}}_{\text{WHERE CE, firing sources only}}.$$

The CE drives ℓ_{sd^*} up exactly where $\alpha = \mathrm{softplus}(\ell)\,\kappa$ concentrates the training-time Dirichlet *and* where the deploy-time sharpened softmax looks, so supervised and RL phases optimize the same parameters with consistent semantics. No new heads, no adapter.

3.3 Data: states visited by the clone itself

Rollouts are generated by the *expert acting in the clone's own action space* (one-hot bundles through the standard decode) against a starter-heavy scripted mix, so the state distribution BC fits is the distribution the warm-started policy will actually visit — a free DAgger-like property; no compounding-error gap on the states that matter. Post-terminal states are masked out; encoded states are stashed on CPU ($\sim$0.6 GB per 500-step rollout at B=64).

3.4 Result

policy	train cost (3070 Ti)	vs random	vs starter	vs greedy	vs medium
best-ever from-scratch RL (v5, 1400 it)	$\approx$20 h	1.00	0.47	—	—
from-scratch v7 smoke (30 it)	6 min	0.80	0.00	0.00	0.00
BC clone, 256/L2 (10 rounds×3 ep)	**144 s**	**1.000**	**0.938**	**0.625**	**0.875**

Table 1: Greedy-deploy GpuEnv scores (win+$\frac{1}{2}$draw, 64 envs, comets on). BC lands at league Elo $\approx$ 900–940 on the anchored scale (random= 0, starter= 600, greedy= 900, medium= 1000; the exact value depends on the deploy greedy: sharpened-softmax 896, categorical argmax 938). Quality gate: a deliberately undertrained clone (64-dim, 2 rounds) reached only dest-acc 0.40 and scored 0.03 vs starter — BC inits are trustworthy only above dest-acc $\approx$ 0.85, reconfirming the v4-era "weak-BC artifact" lesson.

4 WHERE becomes a categorical (the v7 architecture decision)

4.1 The measured failure of PPO on the Dirichlet

Warm-started PPO with the v6 Dirichlet WHERE head *destroys* the BC policy. With all guardrails of §6 active (run `runs/v6/ppoS2`), the fixed-opponent gauntlet decayed 0.755 $\rightarrow$ 0.625 $\rightarrow$ 0.276 $\rightarrow$ 0.104 over 40 iterations while the per-update k_3 KL spiked to 2–9 *despite* a per-minibatch KL hard stop. The mechanism is intrinsic, not a tuning artifact: BC concentrates $\alpha = \mathrm{softplus}(\ell)\kappa$ into near-one-hot rows, and the log-density of a 48-dimensional Dirichlet near a simplex vertex is

so steep that a *single* clipped, LR-warmed optimizer step moves stored-action log-probs by whole nats. PPO's ratio clip bounds each sample's loss contribution, and the KL stop bounds damage per *epoch* — nothing bounds damage per *step* when the density itself is this sharp. (The same property produced the unbounded negative Dirichlet entropy of §5.)

4.2 The replacement

The WHERE head keeps its masked logits ℓ_{sd} (alive + sun-reachability + no-self, **see `rl_math_v6.tex` § dist**) but the distribution over them becomes a per-source **categorical**; the sampled action row is the *one-hot* of the drawn destination, so the action bundle $(B, E, E{+}1)$, the decode, the rollout buffers, the league fingerprint, and the checkpoints are all untouched. Three independent arguments select this design:

1. **Numerics.** $\log \mathrm{softmax}(\ell)_d$ has bounded sensitivity in ℓ; PPO on factored categoricals is its most battle-tested regime. Measured: first-update KL drops from 0.53–2.06 (Dirichlet, from scratch) to 0.008–0.010 (categorical, same setup).

2. **Deploy equivalence.** The v6 *deployed* policy already collapses the Dirichlet to a temperature-sharpened softmax over the same ℓ — effectively one-hot. The continuous "split allocation" expressivity was paid for in training fragility and never used at deploy; the gate-as-rate (fire probability over repeated steps, garrison refilled by production) already delivers multi-target pressure over time.

3. **Search-readiness.** A fully discrete factored action (categorical WHERE × Bernoulli FIRE) is the prerequisite for the planned Gumbel-AlphaZero extension (**see `rl_math.tex` § "search-based policy improvement"** (`sec:search`)); the Dirichlet was the blocker.

The BC loss of §3 is *exactly* the categorical log-likelihood, so the same BC checkpoint warm-starts both variants; `WHERE_DIST='dirichlet'` retains the old path for A/B.

4.3 Attribution: what the Dirichlet did and did not cause

A follow-up 2×2 (from-scratch × {categorical, Dirichlet}, both under the full §6 guardrails, 30 iters, 256/L2) plus a low-LR BC leg separates three phenomena that were previously conflated:

phenomenon	cause	evidence
bootstrap wall (can't beat starter from scratch in reasonable time)	*not* the distribution: credit assignment + opponent gap	scratch-cat gauntlet 0.083, Elo 170; scratch-dir 0.016, Elo 125 (both walled)
first-update slam (launch rate annihilated at it1)	Dirichlet × *unguarded* update; guardrails alone remove it	old code: it1 KL 2.06, lnch 13.5→1.0; guardrailed dir: KL ≤0.024, no slam
post-BC erosion	Dirichlet log-density sharpness; LR-dependent (trust-region misfit)	LR 10^{-4}: gauntlet 0.755→0.104; LR 10^{-5}: 0.719→0.771 (survives)

So the bootstrap wall is *distribution-independent* (it also predates the Dirichlet: the v5 actor's destination head was already categorical and still needed 1400 iterations for a 0.47 starter win rate); BC removes it under *either* distribution. The Dirichlet's real crime is the update fragility, and even that is survivable at 10× reduced LR — but strictly dominated: BC+Dirichlet at LR 10^{-5} reaches gauntlet 0.771 with median KL 0.102 (still 2–3× over target), vs. BC+categorical at LR 10^{-4} reaching 0.812 with KL 0.035 and Elo +161 vs. −9 over the same 40 iterations. You can pay for the Dirichlet with a 10× smaller trust region; there is no reason to.

Mechanism, quantified (CPU probe, `scripts/probe_dist_sensitivity.py`). Over E=48 logit rows, sampled actions from each distribution, identical logits: the Dirichlet's $\|\partial \log \pi(a)/\partial \ell\|_2$ is 5–6× the categorical's at every sharpness regime, and the log-prob movement per unit logit perturbation is 5–10× — which compounds quadratically in KL, matching the ∼100× KL gap measured between the real runs. The decisive asymmetry appears at BC sharpness: the *categorical's* gradient on confident samples collapses to ≈ 0 (a sharp categorical is self-stabilizing), while the Dirichlet's stays ≈ 5 *forever* — it cannot settle, because suppressed components sit at $\alpha = \text{softplus}(-6) \approx 0.003 < 1$, where the density is *singular* at the simplex corner that every sample occupies; the $(\alpha_d - 1) \log x_d$ terms keep injecting $|\log x_d| \approx 14$-scale gradients regardless of how converged the policy is.

5 Entropy bonus: gate-only

The v6 bonus used $H(\text{Bern}(p_s)) + p_s H(\text{Dir}(\alpha_s))$. Differential entropy of a peaked Dirichlet is unboundedly *negative*: after BC it measured $\approx -4{,}000$ per source, so with coefficient 0.005 the "bonus" became a +20 loss term whose gradient actively blurred the BC-learned WHERE head (measured at warmup-end: grad-norm 1380, KL 4.8, Elo $1032 \to 955$). v7 keeps only bounded terms: the gate Bernoulli entropy ($\leq \ln 2$), plus the categorical WHERE entropy ($\leq \ln E$) weighted by `ENT_DIR_COEF` (default 0). Exploration is supplied by the stochastic gate (`GATE_EPS` keeps $p \in [\varepsilon, 1-\varepsilon]$), categorical sampling, and PFSP opponent diversity — not by fighting the BC sharpness.

6 PPO guardrails

Three additions to the v6 update (**see `rl_math_v6.tex` § ppo**); all are off-by-default-compatible knobs in the config cell, defaults chosen for the warm-started regime:

1. **Per-minibatch KL hard stop** (`KL_STOP_MINIBATCH`, default on). The update *breaks mid-epoch* the moment a minibatch's k_3 KL estimate exceeds `KL_HARD_MULT` (1.5) × `KL_TARGET` (0.05). The v6 end-of-epoch check provably cannot prevent the iteration-1 slam (KL = 2.06 happened *within* one epoch).

2. **Linear LR warmup** (`LR_WARMUP_ITERS`): $\eta_t = \eta \cdot \min(1, (t+1)/W)$ over the first W iterations *of a `train()` call* (not global iters), so every fresh-Adam phase (from-scratch, post-BC, resumed-with-fresh-moments) ramps in.

3. **Critic-only warmup** (`VALUE_WARMUP_ITERS`): for the first K iterations the policy-surrogate and entropy terms are multiplied by 0 *and only `val_*` parameters are stepped*. The second clause is load-bearing and was measured the hard way: with a shared trunk, the value loss alone (backpropagating into the trunk) dragged the BC policy by KL 0.15–0.84 per iteration with the policy surrogate completely off. Rationale for the warmup itself: after BC the actor is strong but the critic is random; advantages computed against a random baseline are structured noise, which is exactly what slammed previous warm starts. With the val-only restriction the warmup iterations measure KL $\equiv 0.000$, then the actor unfreezes against a usable baseline.

7 Evidence: the scaling ladder on the 3070 Ti

Protocol: identical recipe at two scales — BC warm-start (10 rounds × 3 epochs) followed by 40 PPO iterations (B=64, T=500, LR warmup 8, value warmup 5, per-minibatch KL stop on,

entropy 0.005 → 0.002), fixed-opponent gauntlet (starter/intermediate/greedy, 64 envs each) every 10 iterations for best-checkpoint selection.

7.1 The decisive A/B at S-scale (256/L2): Dirichlet vs. categorical

Same BC init, same guardrails, same seeds — the only change is the WHERE distribution:

	BC init	it10	it20	it30	it40
gauntlet, Dirichlet (`ppoS2`)	∼0.74	0.755	0.625	0.276	0.104
gauntlet, categorical (`ppoS3`)	∼0.74	0.760	0.802	**0.812**	0.807
league Elo, categorical	938	1050	1092	1087	**1099**
update KL band, Dirichlet	—	0.2–8.9 (spiking, uncontained)			
update KL band, categorical	—	0.025–0.045 (in band, all 40 iters)			

The categorical run *passes the medium anchor* (Elo 1000) by iteration 20 and keeps the KL exactly where `KL_TARGET` asks. Training win rates vs. the medium anchor reach 0.80–0.95 by the back half of the run. This is the first time in the project that RL *improves on* a strong initialization instead of eroding it.

7.2 Scale leg M (512/L4, 13.2M params): same recipe, strictly better

	S: 256/L2 (1.8M)	M: 512/L4 (13.2M)
BC clone vs starter / greedy / medium	0.938 / 0.625 / 0.875	0.906 / **0.703** / **0.953**
gauntlet @ it10 / best of 40	0.760 / 0.812	**0.844** / **0.885**
league Elo, BC → it40	938 → 1099	1073 → **1172**
median update KL (post-warmup)	≈ 0.035	0.044
wall clock per PPO iter (B=64, T=500)	17.1 s	20.8 s
BC phase (10 rounds × 3 epochs)	144 s	168 s

Table 2: The scaling ladder. M dominates S on every quality endpoint at identical hyperparameters and iteration count, with the KL signature unchanged — the recipe transfers across a 7× parameter jump untouched. Crucially, M costs only +22% wall clock per iteration: **the rollout is simulator-bound, so model capacity is nearly free** at these scales — the strongest possible argument for scaling the model further on the A100.

Both legs show the same qualitative dynamics: critic-only warmup (KL ≡ 0, iters 1–5), one contained transient at actor-unfreeze, then a flat in-band KL while Elo climbs ∼25 points per 10 iterations. Neither leg shows the erosion mode of the Dirichlet A/B. The 40-iteration budget was chosen to fit the 3070 Ti's 30-minute experiment ceiling; nothing in either trace suggests saturation at it40 (M's Elo was still rising: 1126 → 1160 → 1180 over the last 15 iterations).

8 Deployment + compute budget

8.1 Kaggle inference budget (measured rule, probed hardware)

`actTimeout` = 1 s per move (`REFERENCE_orbit_wars.json`), 500 steps, `remainingOverageTime` on top. The submission probe measured the Kaggle worker at ∼ 0.032 TFLOP/s fp32 (2 CPU cores, 12.9 GB/s copy bandwidth). Transformer cost per move is $\approx 12d^2L \cdot 48$ MACs:

scale	d/L	params	GFLOP/move	est. ms/move (Kaggle CPU)
S	256/2	1.8M	0.15	5
M	512/4	13.2M	1.21	38
L	768/6	43.8M	4.08	127
XL	1024/8	102.9M	9.66	302

Even XL uses $<1/3$ of the per-move budget; **the deploy budget never binds the architecture choice** up to ∼100M params. Model scale is bounded by *training* compute, not inference.

8.2 Training cost model (measured on the 3070 Ti)

quantity	measured	note
PPO iter, S (256/L2, B=64, T=500)	17.1 s (≈1870 sps)	rollout-dominated
PPO iter, M (512/L4, same batch)	20.8 s (≈1540 sps)	+22% for 7× params
BC round (rollout + 3 supervised epochs)	14–17 s	10 rounds suffice (dest-acc ≥ 0.90)
fixed-opponent gauntlet (3 anchors × 64 envs)	≈2 min	every `CKPT_EVERY` iters
world-pool generation	7 ms/world	negligible (2048 ≈ 15 s)
full S leg (BC + 40 iters + evals)	≈16 min	
full M leg (BC + 40 iters + evals)	≈19 min	

Decision rule applied throughout: any single experiment projecting >30 min on the 3070 Ti is packaged for the A100 instead. Because the workload is simulator-bound, the A100's leverage is *batch* (more parallel envs per iteration) and *capacity* (bigger model at ∼flat iter cost), not raw matmul speed; bf16 autocast + flash SDPA + fused Adam are already in the notebook and the 3070 Ti runs the identical code path.

9 The A100 run

`notebooks/setup1_v7_a100.ipynb` (generated by `scripts/make_a100_nb.py` from the patched v6 notebook, so it carries every fix in this document). Spec:

- **Model**: d=768, L=6 transformer (43.8M params; 127 ms/move on the Kaggle CPU — comfortably deployable). **Batch**: B=256 envs × T=500 (128k transitions/iter, 4× the ladder legs). **Iters**: 600 (additional calls resume via `TRAIN_STATE_PATH`).
- **Recipe**: `BC_ENABLED=True` (Run-All chains BC → Elo calibration → guardrailed PPO), `WHERE_DIST='categorical'`, gate-only entropy (0.005 → 0.002), `VALUE_WARMUP_ITERS=5`, `LR_WARMUP_ITERS=8`, per-minibatch KL stop, PFSP league with snapshots every 25 iters, gauntlet best-ckpt every 25 iters, Elo re-grounding every 500.
- **Projected cost**: simulator-bound scaling from the M leg gives ≈30 s/iter at B=256 on the A100 (≈3× device factor on the element-wise-heavy env step); 600 iters + 24 gauntlets ≈ **6 h**, one Colab session. VRAM ≈5 GB of rollout buffers + model/optimizer — far inside 40 GB; B=512 is the next lever if throughput disappoints.
- **Health signature to watch** (from the ladder): iters 1–5 KL ≡ 0.000; one transient at unfreeze; then KL ∈ [0.02, 0.06], Elo + ∼25/10 iters, gauntlet ≥ BC level (≈0.8) and rising. Any return of KL spikes > 1 or a falling gauntlet means the `WHERE_DIST` flag got flipped or the BC init failed its dest-acc ≥ 0.85 gate.